is relatively rare but not entirely unheard-of, further blurring the lines between the files.

Point 3 relates to the fact that you must call `ConfigureWebHostDefaults()` (which uses an `IWebHostBuilder`) to set up Kestrel and register your `Startup` class. This level of indirection (and the introduction of another builder type) is a remnant of decisions harking back to ASP.NET Core 1.0. For people familiar with ASP.NET Core, this pattern is just one of those things, but it adds confusion when you're new to it.

NOTE For a walk-through of the evolution of ASP.NET Core bootstrapping code, see my blog post at http://mng.bz/pPPK.

Similarly, the lambda-based configuration mentioned in point 4 can be hard for newcomers to ASP.NET Core to follow. If you're new to .NET, lambdas are an extra concept you'll need to understand before you can understand the basics of the code. On top of that, the execution of the lambdas doesn't necessarily happen sequentially; the `HostBuilder` essentially queues the lambda methods so they're executed at the right time. Consider the following snippet:

```
public static IhostBuilder CreateHostBuilder(string[] args) =>
  Host.CreateDefaultBuilder(args)
    .ConfigureLogging(logging => logging.AddSeq())
    .ConfigureAppConfiguration(config => {})
    .ConfigureServices(s => {})
    .ConfigureHostConfiguration(config => {})
    .ConfigureWebHostDefaults(webBuilder =>
    {
        webBuilder.UseStartup<Startup>();
    });
```

The lambdas execute in the following order: